Two-sided independence from Peano arithmetic for a 213-state Turing machine

Abstract

PA213 is a binary Turing machine with 213 working states. It searches for a contradiction in ordinary first-order Peano arithmetic. This article describes the machine, relates its execution to PA proofs, and proves both unprovability claims for its arithmetic nonhalting sentence N:

$$\mathrm{PA} \nvdash N \qquad \text{and} \qquad \mathrm{PA} \nvdash \neg N.$$

The first conclusion follows from consistency of PA and the checked internal implication $\mathrm{PA} \vdash N \to \mathrm{Con(PA)}$. The second uses soundness of PA in the standard natural numbers, a theorem of ordinary classical mathematics. The machine-specific proof and the independence corollaries are checked in Lean 4 on top of Foundation. Their axiom audit reports only `propext`, `Classical.choice`, and `Quot.sound`.

Outline

Section 2 describes the machine and the program it runs. Section 3 writes down the arithmetic sentence N. Section 4 connects machine runs to PA proofs in both directions. Sections 5 and 6 prove the two halves of independence. Section 7 explains why searching for a proof of $0 = 1$ is the design already used. Section 8 lists the Lean results. Figure 1 shows how a refutation leads to a halting trace.

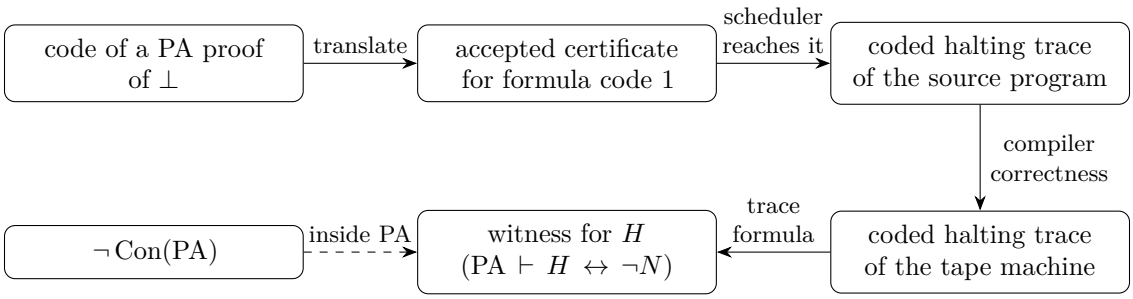


Figure 1: The proof of $\mathrm{PA} \vdash N \to \mathrm{Con(PA)}$. Each arrow is a checked Lean construction on codes, uniformly in every model of PA; the dashed arrow summarizes the composite inside PA. In a nonstandard model these codes can have nonstandard lengths and need not describe actual finite proofs or runs. In $\mathbb{N}$, they describe genuine finite objects. The converse, that consistency gives actual nonhalting, is a separate argument: soundness of the object calculus, the stack invariant, and positive progress of the simulation (Section 4).

1 Main theorem and preliminaries

Busy Beaver values are defined by Radó [4]. Yedidia and Aaronson [5] constructed a 7,910-state machine whose nonhalting is unprovable in ZFC if ZFC is consistent; their argument for its actual

nonhalting uses stronger set-theoretic assumptions. They also asked how small such machines can be. PA213 addresses this question for Peano arithmetic. Its proof is formalized in Lean 4 on top of the Foundation library [6]; the source is [1]. We make no claim that 213 states is minimal.

Let PA be ordinary first-order Peano arithmetic with full induction. The formalization uses Foundation's theory `Peano`, in the language of zero, one, addition, multiplication, equality, and order. Write $\mathrm{PA} \vdash A$ when a finite formal PA proof of A exists, and $\mathbb{N} \models A$ when the sentence is true in the standard natural numbers. Truth alone does not imply provability.

Let M be the exact machine specified in Section 2, and let H be the repository's arithmetic sentence asserting that M has a finite halting trace from the blank tape. Define $N := \neg H$. In particular, $\neg N$ is classically equivalent to H.

Theorem 1.1 (Independence of the nonhalting sentence)**.** *The machine M does not halt from the blank tape, $\mathbb{N} \models N$, and*

$$\mathrm{PA} \nvdash N \quad \textit{and} \quad \mathrm{PA} \nvdash \neg N.$$

We work in ordinary classical mathematics, where $\mathbb{N} \models \mathrm{PA}$ and hence PA is consistent (Lemma 6.1). This is not a PA proof of its own consistency or soundness. The two directions use different hypotheses:

- *Consistency of PA* gives nonhalting, $\mathbb{N} \models N$, and $\mathrm{PA} \nvdash N$ (Proposition 5.1).
- *Soundness of PA in* $\mathbb{N}$ gives $\mathrm{PA} \nvdash \neg N$ (Proposition 6.2).

The original Lean endpoint assumes consistency. The two-sided module discharges both hypotheses with Foundation's proof that $\mathbb{N}$ models PA. No machine bytes, existing theorem statements, or assumptions of the five original audited roots are changed.

2 Definition of PA213

The tape is infinite in both directions, has alphabet $\{0, 1\}$, and is initially all zero. In the published table the head starts at the origin in state 0. The working-state set is

$$Q = \{0, 1, \ldots, 214\} \setminus \{122, 214\}, \qquad |Q| = 213.$$

State 215 is the halt label and is not a working state. Each pair in $Q \times \{0, 1\}$ has one transition, giving exactly 426 rows. A row (q, b, b', D, q') means: read b in state q, write b', move one cell in direction D, and enter q'. For example, the first row is

$$(0, 0, 1, R, 1).$$

The full table defines M: `Lean/data/pa213.tm`. Its SHA-256 digest is

`b332b36d8ed9c909c51df27727a2c68a6890c122e102a4e7a6d411a483094989`.

The physical simulation proof uses equivalent sparse labels, with start label 74 and halt label 254. The arithmetic sentence uses the published labels and exactly the 426 active transitions. Relabeling does not add states or change halting behavior.

2.1 Refutation search

The machine runs a register program that enumerates finite lists of proof commands in a small first-order calculus. It executes the commands in order and halts when they derive a fixed false formula. That calculus proves a contradiction if and only if PA does. The direction from PA refutations to calculus refutations is arithmetized inside PA. The other direction is an external theorem, which suffices for this proof. The following sections describe the calculus, the command interpreter, the enumeration, and the compilation to a tape machine.

2.2 The numeric proof calculus

Formulas are coded by natural numbers using the Cantor pairing $\langle a, b\rangle = \binom{a+b+1}{2} + b$. Variables are number labels. If A, B are the formulas decoded from a, b, the decoder uses

$$\begin{aligned} 0 &\mapsto \top, \\ 4\langle a, b\rangle &\mapsto A \to B \qquad (a + b > 0,\ b \neq 1), \\ 4\langle a, 1\rangle &\mapsto \neg A, \\ 4\langle a, b\rangle + 1 &\mapsto x_b \in x_a, \\ 4\langle a, b\rangle + 2 &\mapsto \forall x_a\, B, \\ 4\langle a, b\rangle + 3 &\mapsto x_a = x_b. \end{aligned}$$

In particular, code 1 decodes to $x_0 \in x_0$, not to a primitive logical falsity symbol. The numeric constructor for negation is `wn a = wim a 1`, and the decoder treats that code specially as logical negation. These interpretations agree in the intended arithmetic models because self-membership is false there. The encoding is not injective on named formulas (`NativeDecode.lean`). The only nonlogical relation symbols are $\in$ and $=$.

The rules are modus ponens and generalization. The axioms are nine schemas (`ActualAXCode.lean`): quantifier distribution and exchange, instantiation, existence of a witness for $x_a = x_b$, congruence of $=$ with $\in$, transitivity of $=$, the single implicational axiom $((p \to q) \to r) \to ((r \to p) \to (s \to p))$ of Łukasiewicz (with the numeric restriction $u \leq a$ in `rho8 a b c u`), and a guarded *pointed separation* axiom: for a set x_a, a point x_b, and a formula φ, some x_c contains the members of x_a satisfying φ, only elements satisfying φ, and x_b if $\varphi(x_b)$.

To prove source soundness, the formalization first derives the literal AX formulas in the supporting `NativeCalculus`. The two presentations are not definitionally identical; their connection is proved in `ActualAXBridge.lean`. The two directions connecting the object calculus to PA use different proofs:

- *Soundness.* Interpret $y \in x$ as "bit y of x is 1" (Ackermann's coding of hereditarily finite sets) inside any model of PA. Every derivable formula is true under this interpretation. The formula $x_0 \in x_0$ is false because $y \in x$ implies $y < x$. By completeness, an actual finite derivation of code 1 gives PA $\vdash \bot$. This is an external theorem about finite derivations, not an internal translation of proof codes (`NativeSoundness.lean`, `native_refutation_PA_bottom`).

- *Completeness for refutations, inside PA.* Numbers are interpreted as finite von Neumann–style ordinals ($0 = \emptyset$, $Sa = a \cup \{a\}$), with equality read as order isomorphism and induction as ordinal induction using separation. A PA proof of $\bot$ is first translated into an equivalent presentation of PA with successor, then into a certificate accepted by the calculus. The translation works on proof codes in every model of PA, including nonstandard codes (`FoundationNativeRefutationTransport.lean`).

Deriving code 1 is therefore the contradiction the machine looks for.

2.3 Command semantics

The interpreter state consists of four numbers (t, w, d, h). The registers t and h hold formula codes, w encodes a stack by iterated pairing, and d is a label offset. There are three commands (`CommandSemantics.lean`, `ActualCommandSemantics.lean`):

Command	Effect
shift	If $t + h = 1$, halt. Otherwise $t \leftarrow h$ and pop the top of w into h.
mp x	Put $y = d + x$. If h codes $t \to y$, then $t \leftarrow y$ (modus ponens) and push 0; otherwise $t \leftarrow 0$. In both cases $d \leftarrow y$ and $h \leftarrow 0$.
ax c x y	With parameters $a = d + x$ and $b = h + y$, push eight axiom instances and the generalization $\forall x_d \, \varphi_h$ of the formula coded by the incoming h, together with the old t; set t to the separation instance (or to truth if c collides with a or b) and $d, h \leftarrow 0$.

The checked invariant `Invariant` combines `TopSound`, which says that t and h are derivable, with `WrapperStack.Safe`, which says the same for every stack entry above a fixed initial tail. A shift halts only if $t + h = 1$, so one of the two derivable registers equals 1, and code 1 is derivable (`shift_halt_refutation`). This is the only halting instruction in the program.

2.4 Enumeration of command sequences

A number n decodes to a command list word(n) by repeated unpairing (`Enumeration.lean`). The program executes word(0), word(1), word(2), ... in order, keeping the interpreter state between words. Completeness of the search follows from two checked facts:

- every finite command list is a prefix of some word(n);
- there is a fixed "halt word" (reset, then the compiled refutation, then shift) that halts from *every* state when a refutation exists.

The stack also retains enough derivable entries to continue: ax pushes ten entries and shift pops one. A counting argument gives every prefix of the schedule a net weight of at least -3 against an initial depth of 9. If the calculus is consistent, every command can execute and the program never halts.

2.5 Simulation by a binary Turing machine

The enumeration and interpreter are written as a register program with nine unbounded registers and 3,947 instruction sites. The instructions are increment, decrement-or-branch-on-zero, structured break and continue, and a single halt; control returns to the first site after the last (`Syntax.lean`).

On the tape, a configuration of the register program is a *frame*: a marker, the binary path of the current site, padding, and the register file with each value v written in unary as $0\,1^{v+1}$ (`RootFrame.lean`). A *runtime contract* states that from the frame of a site, the tape machine reaches the frame of the correct successor site in a positive number of steps. There is one contract per site and a second one for each of the 1,187 decrement sites, giving $3{,}947 + 1{,}187 = 5{,}134$ contracts, each checked by a certified symbolic executor. Starting from the blank tape, the machine runs exactly 64,764 steps to a frame of the register program. That frame is the state the program reaches 159 steps after a fixed seed state, the entry of the initial command ax 0 0 0.

Remark 2.1. *Many supporting modules use the namespace* `PA215`*. Their register program and proof libraries were first built for a 215-state machine and reused unchanged for PA213. Keeping their names lets the source hashes be checked against the original files.*

3 Arithmetization of halting

A configuration is represented by a state q and two natural numbers l, r. The low binary digit of r is the scanned cell; subsequent digits describe the tape to its right. The low digit of l is the cell immediately to the left of the head. Unrepresented high digits are zero. This encodes every configuration reached in finitely many steps from a blank tape.

For a transition reading b and writing w, a right move has the relation

$$q = q_0 \ \wedge \ q' = q_1 \ \wedge \ \exists t\,(r = 2t + b \ \wedge \ l' = 2l + w \ \wedge \ r' = t).$$

A left move has the relation

$$\begin{aligned} q = q_0 \ \wedge \ q' = q_1 \ \wedge \ \exists t, u, e\,((e = 0 \vee e = 1) \wedge l = 2u + e \wedge r = 2t + b \\ \wedge\, l' = u \wedge r' = 2(2t + w) + e). \end{aligned}$$

Let $F(q, l, r, q', l', r')$ be the disjunction of these relations over the 426 rows. The relations are defined in `StepFormula.lean`, restricted to the active rows by `ExactHaltingFormula.lean`.

Using the repository's arithmetic relation for sequence access, write (q_i, l_i, r_i) for the ith entries of three coded sequences. Then H is the following formula, with sequence notation expanded into arithmetic:

$$\exists n, \vec{q}, \vec{l}, \vec{r}\ \begin{bmatrix} q_0 = 0 \wedge l_0 = 0 \wedge r_0 = 0 \wedge q_n = 215 \\ \wedge\, \forall i < n\ F(q_i, l_i, r_i, q_{i+1}, l_{i+1}, r_{i+1}) \end{bmatrix}. \tag{1}$$

The arrows denote number codes, not second-order sequence variables. The Lean definition explicitly quantifies the entries and relates them to the codes using `nthDef`. Equation (1) abbreviates that definition of the exact sentence used in the proof.

The checked adequacy theorem establishes

$$\mathbb{N} \models N \quad \Longleftrightarrow \quad M \text{ does not halt from its blank initial configuration.} \tag{2}$$

An auxiliary encoding has two padding states and 430 rows. An invariant valid in every PA model proves those states unreachable, giving the required equivalence with the exact 426-row sentence.

4 Correctness of the refutation search

The external simulation proves nonhalting. A separate implication inside PA lets us apply the second incompleteness theorem.

4.1 Nonhalting from consistency

The source program has 3,947 instruction sites. A checked startup of 64,764 tape steps reaches a frame corresponding to a 159-step prefix of that source. The compilation proof uses 5,134 runtime contract records. At each reachable instruction frame it proves that a positive, finite number of tape steps implements the next source instruction.

If PA is consistent, soundness of the object calculus rules out a derivation of code 1, so the search continues. The simulation takes at least one tape step per source instruction: after k instructions,

at least k tape steps have occurred. If the machine halted after t tape steps, simulation of another sufficiently long finite source prefix would give a run longer than t from the same deterministic initial configuration, a contradiction. The checked theorem is

$$\mathrm{Con(PA)} \quad \Longrightarrow \quad M \text{ does not halt.} \tag{3}$$

Here Con(PA) is read externally as absence of an actual PA refutation. The source-to-machine theorem is `consistent_machine_nonhalting` in `MachineNonhalting.lean`. Together with (2), it gives $\mathbb{N} \models N$ under consistency.

4.2 Internal consistency implication

Choose the standard arithmetized PA proof predicate, and put

$$\mathrm{Prov}_{\mathrm{PA}}(x) := \exists p\, \mathrm{Prf}_{\mathrm{PA}}(p, x), \qquad \mathrm{Con(PA)} := \neg\, \mathrm{Prov}_{\mathrm{PA}}(\ulcorner \bot \urcorner).$$

The following argument is formalized in every model of PA. A code for a PA refutation is translated into a refutation code accepted by the actual numeric checker. The scheduler reaches that code, and its source execution is converted to a tape trace. Adding the startup prefix gives a trace from the blank initial configuration. In particular,

$$\mathrm{PA} \vdash \mathrm{Prov}_{\mathrm{PA}}(\ulcorner \bot \urcorner) \to H, \qquad \text{hence} \qquad \boxed{\mathrm{PA} \vdash N \to \mathrm{Con(PA)}.}$$

The proof first uses the auxiliary trace, then eliminates its padding states. First-order completeness gives a PA derivation because the implication holds in all PA models, including nonstandard ones. Equivalence in the standard model alone would not suffice.

The relevant source files are `FoundationNativeRefutationTransport.lean`, `InternalActualRefutationHalting.lean`, and `NonhaltingUnprovability.lean`. The argument uses only the displayed internal implication. Its converse is not part of the frozen endpoint.

5 Unprovability of nonhalting

Proposition 5.1. *If PA is consistent, then* $\mathrm{PA} \nvdash N$.

Proof. If $\mathrm{PA} \vdash N$, the internal implication of Section 4 gives $\mathrm{PA} \vdash \mathrm{Con(PA)}$ by modus ponens. Gödel's second incompleteness theorem [2] for the standard PA provability predicate forbids this when PA is consistent. In Lean, the internal implication is stated for the sentence `PA215.PA.consistencySentence`, and the incompleteness step (`unprovable_of_internal_consistency_implication`) is applied to that same sentence, so no identification of two consistency formulas is needed. □

Abbreviate $\mathrm{Prov}_{\mathrm{PA}}(\ulcorner A \urcorner)$ by $\Box A$. Standard provability satisfies:

$$\begin{aligned} &\mathrm{PA} \vdash A \implies \mathrm{PA} \vdash \Box A, \\ &\mathrm{PA} \vdash \Box(A \to B) \to (\Box A \to \Box B), \\ &\mathrm{PA} \vdash \Box A \to \Box\Box A. \end{aligned}$$

The diagonal lemma supplies G with $\mathrm{PA} \vdash G \leftrightarrow \neg\Box G$. If PA proved G, it would prove both $\Box G$ and $\neg\Box G$, contrary to consistency. Internally, the displayed derivability conditions give

$$\Box G \to \Box\neg\Box G, \qquad \Box G \to \Box\Box G, \qquad \Box G \to \Box\bot.$$

Thus PA proves $\neg\Box\bot \to \neg\Box G$, and hence $\mathrm{Con}(\mathrm{PA}) \to G$. A PA proof of consistency would therefore give a PA proof of G, which was just ruled out. The project imports the formal second incompleteness theorem through `Foundation.lean`; the diagonal lemma and derivability conditions belong to that general result and require no further facts about PA213.

6 Unprovability of halting

Lemma 6.1 (Soundness of PA in the standard model)**.** *In ordinary classical set theory, for every arithmetic sentence A,*

$$\mathrm{PA} \vdash A \quad \Longrightarrow \quad \mathbb{N} \models A.$$

In particular, PA is consistent.

Proof. Interpret zero, one, addition, multiplication, and order by their usual operations and relation on $\mathbb{N}$. The noninduction axioms of PA hold in this structure. For any first-order formula $\varphi(x, \vec{y})$ and any assignment of natural numbers $\vec{a}$ to its parameters, consider its induction instance. If $\varphi(0, \vec{a})$ holds and $\varphi(n, \vec{a})$ implies $\varphi(n+1, \vec{a})$ for every n, ordinary induction in the surrounding mathematics gives $\varphi(n, \vec{a})$ for every natural number n. Thus every induction axiom is true under every assignment, so $\mathbb{N} \models \mathrm{PA}$.

Logical axioms are valid, and the inference rules preserve truth under all assignments: in particular, modus ponens preserves truth and generalization preserves validity with the usual variable restrictions. Induction on the length of a finite derivation therefore proves that every PA theorem is true in $\mathbb{N}$. Since $\bot$ is false in $\mathbb{N}$, it cannot be a PA theorem. □

The proof takes place in the surrounding set theory, where $\mathbb{N}$ is a set-sized structure. PA has not proved its own soundness or consistency, so the second incompleteness theorem still applies.

Proposition 6.2. *If PA is sound in $\mathbb{N}$, then $\mathrm{PA} \nvdash \neg N$; equivalently, $\mathrm{PA} \nvdash H$.*

Proof. Soundness implies consistency. By (3) and (2), the machine does not halt and $\mathbb{N} \models N$. If $\mathrm{PA} \vdash \neg N$, soundness would give $\mathbb{N} \models \neg N$, contradicting $\mathbb{N} \models N$. Classical double-negation elimination identifies provability of $\neg N = \neg\neg H$ with provability of H. □

Proof of Theorem 1.1. Soundness supplies consistency. The checked nonhalting and adequacy results give actual nonhalting and truth of N. Proposition 5.1 excludes a PA proof of N; Proposition 6.2 excludes a PA proof of its negation. Lemma 6.1 establishes the soundness premise in the stated metatheory, completing both directions. □

6.1 Consistency and soundness assumptions

Consistency alone says that a theory has no derivation of a contradiction. It does not imply that the theory's existential arithmetic theorems have standard witnesses. A classical proof of "there exists a halting trace" need not yield a specific standard trace just by inspecting that proof. We use soundness for the second direction.

For example, if PA is consistent, then $\mathrm{PA} + \neg\,\mathrm{Con}(\mathrm{PA})$ is consistent: otherwise the deduction theorem would give $\mathrm{PA} \vdash \mathrm{Con}(\mathrm{PA})$, contradicting the second theorem. Yet this extension asserts the existence of a PA refutation when there is no standard such refutation. This extension is consistent but unsound; the example does not say that PA itself proves $\neg\,\mathrm{Con}(\mathrm{PA})$.

For a conventional Σ_1 halting formula, Σ_1-soundness is enough to exclude a proof of false halting. To use that weaker hypothesis for this exact sentence, one must also establish its PA-provable

equivalence to the chosen Σ_1 presentation. The proof above uses full soundness and therefore needs no additional syntactic normalization claim about `ExactHaltingFormula.traceSentence`.

Rosser's theorem gives two-sided undecidable sentences from consistency alone by comparing proofs of a sentence with earlier proofs of its negation [3]. A Rosser-based search would require a different specification and verified implementation before it could be attributed to a concrete small machine. No 213-state bound for such a modification is asserted here.

7 Choice of refutation target and state complexity

For an ideal exhaustive proof search $M_\perp$, the intended equivalence is

$$M_\perp \text{ halts} \iff \exists p\, \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner 0 = 1 \urcorner) \iff \neg\, \mathrm{Con}(\mathrm{PA}).$$

Its *nonhalting* therefore expresses consistency. PA proves $0 \neq 1$, so a proof of $0 = 1$ gives a contradiction. Conversely, a contradiction gives $0 = 1$ by classical logic. The two finite proof transformations can be arithmetized in PA.

More generally, replacing falsity by a fixed B has the same property when $\mathrm{PA} \vdash \neg B$. Merely knowing externally that B is false does not suffice: a false sentence may itself be undecidable in PA. For this reduction, "known false" must mean "PA-refutable".

PA213 already uses this design: it halts when it derives the fixed refutable formula with code 1, namely $x_0 \in x_0$, and its halting test is the single comparison $t + h = 1$ (Section 2). The target is already a contradiction, not a sentence such as $\neg\, \mathrm{Con}(\mathrm{PA})$. The object calculus has no arithmetic symbols, so $0 = 1$ is not one of its formulas. A PA-refutable target would pass through the same translation. Changing the target alone does not establish a state reduction: logically equivalent proof targets can have different compiled state counts. A reduction would require a concrete revised checker, scheduler, encoding, or compiler, followed by a new transition table and corresponding proofs. No such reduction is established here, and 213 states is not claimed to be minimal.

8 Formal verification and reproducibility

The original machine and proof sources used here are from commit

`034d08df73b94e9f4036fc5af74b694eb73afc67`.

The two-sided extension described below was added after that snapshot; its files are named by repository path. The frozen formalization uses Lean 4.35.0. Its endpoint audit reports `propext`, `Classical.choice`, and `Quot.sound`. Its five audited roots retain their original scope:

Theorem (namespace omitted)	Checked conclusion
`consistent_machine_nonhalting`	Consistency implies physical nonhalting.
`nonhalting_true`	Consistency implies $\mathbb{N} \models N$.
`pa_proves_nonhalting_consistency`	$\mathrm{PA} \vdash N \to \mathrm{Con}(\mathrm{PA})$.
`nonhalting_unprovable`	Consistency implies $\mathrm{PA} \nvdash N$.
`final_endpoint`	State and row counts, nonhalting, truth, and the preceding unprovability under consistency.
Two-sided extension, namespace `PA213.PA.Independence`	
`nonhalting_irrefutable`	Consistency implies $\mathrm{PA} \nvdash \neg N$, via soundness in $\mathbb{N}$.
`two_sided_independence`	$\mathrm{PA} \nvdash N$ and $\mathrm{PA} \nvdash \neg N$, with no premises.
`machine_nonhalting`	The machine does not halt from the blank tape, with no premises.

The added module `Lean/metamathematics/PA213PA/TwoSidedIndependence.lean` proves the additional results. Foundation's `Arithmetic.models_Peano` proves that $\mathbb{N}$ models PA; `Theory.Proof.sound` makes any PA proof of $\neg N$ contradict `nonhalting_true`. Foundation also supplies PA consistency, so the theorems `two_sided_independence` and `machine_nonhalting` have no explicit premises. This reasoning takes place in Lean's metatheory, not inside PA.

All three added declarations type-check against the built dependencies and use exactly `propext`, `Classical.choice`, and `Quot.sound`, with no `sorryAx`. The default build includes them, and the audit checks all eight roots. The extended report is `evidence/publication/independence-axioms.txt`; the original five-root report and frozen receipt remain unchanged.

From the repository root, run `bash scripts/check.sh` for data and publication checks. After installing the pinned Lean toolchain and dependencies as described in `docs/reproduction.md`, run `bash scripts/check.sh -full` to compile and audit the endpoint. The precise statements are in `docs/result.md`; the extended axiom report is `evidence/publication/independence-axioms.txt`. To build the added module on its own, run

```
python3 Lean/scripts/build_serial.py -metamathematics
-target PA213PA.TwoSidedIndependence
```

from the repository root. Then run `python3 Lean/metamathematics/scripts/check_axioms.py`.

References

[1] *Nonhalting and PA-unprovability for a 213-state Turing machine*. Source snapshot identified above. https://github.com/jamesyc/BusyBeaver.

[2] Kurt Gödel. *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I*. Monatshefte für Mathematik und Physik 38 (1931), 173–198. https://doi.org/10.1007/BF01700692.

[3] J. Barkley Rosser. *Extensions of some theorems of Gödel and Church*. Journal of Symbolic Logic 1 (1936), 87–91. https://doi.org/10.2307/2269028.

[4] Tibor Radó. *On non-computable functions*. Bell System Technical Journal 41 (1962), 877–884. https://doi.org/10.1002/j.1538-7305.1962.tb00480.x.

[5] Adam Yedidia and Scott Aaronson. *A relatively small Turing machine whose behavior is independent of set theory*. Complex Systems 25 (2016), 297–327. https://arxiv.org/abs/1605.04343.

[6] FormalizedFormalLogic. *Foundation: formalized mathematical logic in Lean 4*. https://github.com/FormalizedFormalLogic/Foundation.